

Relatório do Projeto — Simulador de Linguagens Formais e Autômatos

1. Descrição do projeto

O projeto consiste em uma **aplicação web voltada ao estudo de linguagens formais**. Ela apresenta três formas de representação:

- **Expressões regulares;**
- **Autômatos finitos;**
- **Gramáticas.**

Em cada página, o usuário pode definir uma representação e observar seu funcionamento por meio de **testes ou simulações**.

O foco deste relatório é a implementação dos **autômatos finitos**. A aplicação permite:

- Construir um **Autômato Finito Determinístico (AFD)** ou um **Autômato Finito Não Determinístico (AFND)**;
- Cadastrar estados e transições;
- Visualizar graficamente o autômato;
- Verificar se determinadas palavras são aceitas.

O reconhecimento pode ser realizado de três formas:

1. Teste de uma única palavra;
2. Teste de várias palavras simultaneamente;
3. Simulação passo a passo.

O projeto foi desenvolvido utilizando **JavaScript, HTML e CSS**. Um servidor **Express** é responsável por disponibilizar as páginas da aplicação, enquanto a biblioteca **Cytoscape** é utilizada para desenhar graficamente os autômatos.

2. Máquina escolhida: Autômato Finito

Um **autômato finito** é formado por estados e transições. O processamento começa em um **estado inicial** e realiza a leitura da palavra símbolo por símbolo.

Ao final da leitura, a palavra é considerada **aceita** caso o processamento termine em um **estado final**.

O simulador trabalha com dois tipos de autômato:

AFD — Autômato Finito Determinístico

No AFD:

- Para cada símbolo, um estado pode possuir **no máximo uma transição de saída**;
- Não são permitidas **transições ϵ** ;
- Para uma determinada entrada, existe apenas um caminho possível de processamento.

AFND — Autômato Finito Não Determinístico

No AFND:

- Podem existir diferentes caminhos para o mesmo símbolo;
- São permitidas **transições ϵ** ;
- Uma transição ϵ permite mudar de estado **sem consumir nenhum símbolo da palavra**.

Na interface, o usuário pode definir:

- Estado inicial;
- Estados finais;
- Estado de origem da transição;
- Estado de destino;
- Símbolos associados à transição.

Antes de alterar o autômato, a aplicação realiza a **validação das informações fornecidas**.

3. Técnicas utilizadas na implementação

3.1 Representação por lista de adjacência

O modelo armazena os estados utilizando uma estrutura semelhante a uma **lista de adjacência**.

Cada estado contém:

- Seu nome;
- A indicação de que é um estado inicial;
- A indicação de que é um estado final;
- Uma lista contendo suas transições de saída.

Cada transição registra:

- Estado de origem;
- Estado de destino;
- Símbolos associados à transição.

Essa estrutura representa diretamente o diagrama do autômato.

Durante a simulação, o programa consulta as transições disponíveis nos estados atuais para determinar quais são os **próximos estados possíveis**.

3.2 Validação das regras

O modelo possui diversas validações para impedir a criação de autômatos inconsistentes.

Entre as principais regras implementadas estão:

- Estados não podem possuir nomes vazios;
- Não podem existir estados com nomes repetidos;
- Apenas um estado pode ser definido como inicial;
- A origem de uma transição deve existir;
- O destino de uma transição deve existir;
- Os símbolos informados devem ser válidos.

Quando o autômato selecionado é um **AFD**, também são verificadas regras adicionais:

- Não são permitidas transições ϵ ;

- Não podem existir duas transições de saída para o mesmo símbolo a partir de um mesmo estado.

Ao tentar alterar o tipo de um **AFND para AFD**, o programa verifica se todas as transições já cadastradas respeitam essas restrições antes de realizar a alteração.

3.3 Reconhecimento de palavras

A simulação começa pelo **estado inicial**.

Para cada símbolo presente na palavra, o programa analisa todos os estados atualmente ativos e reúne os possíveis estados de destino.

De forma simplificada, o processo segue a sequência:

```
Estado inicial
    ↓
Lê um símbolo
    ↓
Procura as transições possíveis
    ↓
Obtém os próximos estados
    ↓
Repete até terminar a palavra
    ↓
Verifica se algum estado atual é final
```

Ao término da leitura, o programa verifica se pelo menos um dos estados alcançados está marcado como **estado final**.

Para permitir o funcionamento tanto de AFDs quanto de AFNDs, o algoritmo trabalha com um **conjunto de estados atuais**, mesmo quando existe apenas um caminho possível.

Dessa forma, o mesmo algoritmo básico pode ser utilizado para os dois tipos de autômato.

3.4 Cálculo do fecho ϵ

Nos AFNDs, uma transição ϵ pode ser percorrida sem que nenhum símbolo da palavra seja consumido.

Por esse motivo, o programa calcula o chamado **fecho ϵ** .

O fecho ϵ representa o conjunto de estados que podem ser alcançados a partir de determinado estado utilizando **zero ou mais transições ϵ** .

O cálculo é realizado:

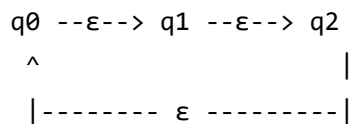
1. Antes da leitura do primeiro símbolo;
2. Após o processamento de cada símbolo da palavra.

O algoritmo mantém:

- Uma lista de estados que ainda precisam ser analisados;
- Um conjunto de estados já encontrados.

Essa abordagem impede que estados sejam processados repetidamente em situações que envolvam **ciclos de transições ϵ** .

Por exemplo:



Mesmo existindo um ciclo, cada estado é registrado apenas uma vez no conjunto do fecho ϵ .

3.5 Simulação visual

A biblioteca **Cytoscape** é utilizada para transformar os estados e transições cadastrados em um **diagrama visual do autômato**.

O sistema diferencia visualmente:

- Estados comuns;
- Estado inicial;
- Estados finais;

- Transições entre estados.

Na opção de simulação **passo a passo**, também são destacados os estados que podem estar ativos após a leitura de cada símbolo.

Essa representação facilita a compreensão do funcionamento do autômato, principalmente em casos envolvendo **AFNDs**, nos quais vários caminhos podem existir simultaneamente.

4. Estruturação do código

O código do projeto foi dividido de acordo com a responsabilidade de cada componente.

Arquivo / Diretório	Responsabilidade
src/models/automato.js	Armazena estados e transições, realiza validações e executa o reconhecimento de palavras.
src/controllers/representacao2.js	Recebe as ações do usuário, atualiza a interface e apresenta os resultados das simulações.
src/views/representacao2/	Contém os arquivos HTML e CSS relacionados à página de autômatos.
src/index.js	Configura o servidor Express e as rotas responsáveis pelas três representações.

As outras duas representações do sistema seguem uma organização semelhante, possuindo seus próprios arquivos de **modelo, controlador e interface**.

Essa separação facilita:

- A manutenção do código;
- A localização das regras relacionadas aos autômatos;
- A alteração da interface sem modificar diretamente a lógica interna;
- A evolução futura do sistema.

5. Avaliação da qualidade da solução

A solução desenvolvida cumpre adequadamente sua finalidade **didática**.

O usuário consegue:

- Criar estados;
- Criar e corrigir transições;
- Escolher entre AFD e AFND;
- Visualizar graficamente o autômato;
- Realizar o reconhecimento de palavras;
- Acompanhar a execução passo a passo.

A validação das transições também impede que configurações incompatíveis com as regras de um **AFD** sejam cadastradas.

Além disso, a execução passo a passo facilita a compreensão do comportamento dos **AFNDs**, principalmente quando existem múltiplos estados possíveis durante o processamento.

Limitações identificadas

Apesar de atender aos requisitos principais, alguns pontos podem ser melhorados.

Cobertura de testes

Os testes automatizados existentes atualmente verificam principalmente o modelo relacionado às **gramáticas**.

Ainda não existem testes automatizados específicos para o modelo de autômatos.

Seria interessante adicionar casos de teste envolvendo:

- Palavra vazia;
- Ciclos com transições ϵ ;
- Múltiplos caminhos em AFND;
- Palavras aceitas e rejeitadas;
- Alteração de AFND para AFD;
- Transições duplicadas;
- Estados finais múltiplos.

Dependência para geração do diagrama

A página utiliza a biblioteca **Cytoscape** carregada por meio de uma URL externa.

Dessa forma, a visualização gráfica depende da disponibilidade dessa biblioteca no navegador e também do acesso ao recurso externo.

Uma possível melhoria seria disponibilizar a biblioteca diretamente dentro do projeto.

Escopo das gramáticas

A página de gramáticas é apresentada na navegação como **“Gramáticas Regulares”**.

Entretanto, o modelo permite produções contendo vários símbolos e não terminais em diferentes posições.

Dessa forma, a implementação atual não delimita de maneira totalmente precisa quais formatos de produção pertencem exclusivamente às gramáticas regulares.

Esse aspecto poderia ser revisado em uma versão futura para tornar a implementação mais compatível com a definição formal utilizada na disciplina.